

Data Preprocessing and Visualisation

Aim

To perform **data preprocessing and visualization** on a sales dataset using Python in order to handle missing values, summarize key statistics, and represent sales trends and relationships through various plots.

Procedure

1. Import the required Python libraries such as pandas, numpy, matplotlib, and seaborn for data analysis and visualization.
2. Load the dataset into a pandas DataFrame using the `read_csv()` function.
3. Examine the dataset using `head()` function and handle the missing values by replacing them or dropping them.
4. Generate summary statistics using the `describe()` function to understand the distribution and range of numerical attributes.
5. Group the dataset by product and calculate the total sales and quantity using the `groupby()` and `agg()` functions for summarized insights.
6. Plot a **bar chart** showing total sales for each product to visually compare product performance.
7. Convert the Date column into datetime format and plot a **line chart** of total sales over time to analyze sales trends.
8. Create a **pivot table** to compare regional and product-wise sales, compute the **correlation matrix**, and visualize it using a **heatmap** to identify relationships between variables.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
fp='sales_data.csv'
df=pd.read_csv(fp)
print(df.head())
```

	Date	Product	Sales	Quantity	Region
0	01-01-2023	Product A	200	4	North
1	02-01-2023	Product B	150	3	South
2	03-01-2023	Product A	220	5	North
3	04-01-2023	Product C	300	6	East
4	05-01-2023	Product B	180	4	West

```
print(df.isnull().sum())
```

```
Date      0
Product    0
Sales      0
Quantity   0
Region     0
dtype: int64
```

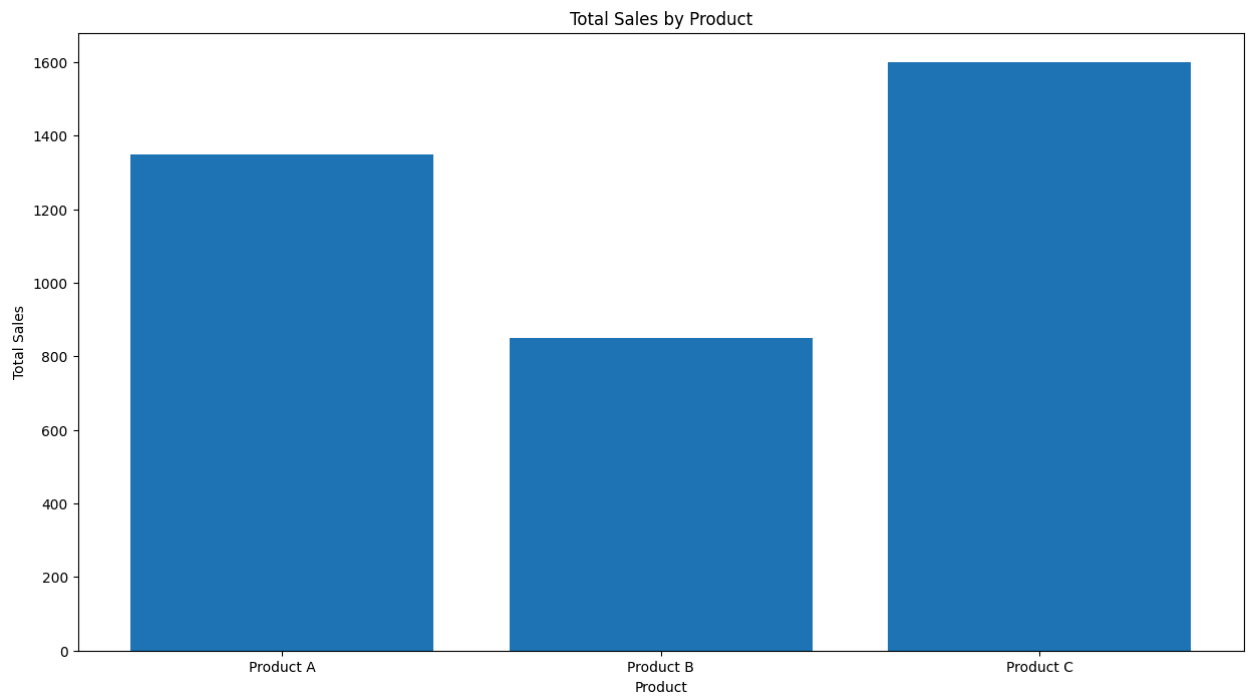
```
print(df.describe())
```

	Sales	Quantity
count	16.000000	16.000000
mean	237.500000	5.375000
std	64.031242	1.746425
min	150.000000	3.000000
25%	187.500000	4.000000
50%	225.000000	5.500000
75%	302.500000	7.000000
max	340.000000	8.000000

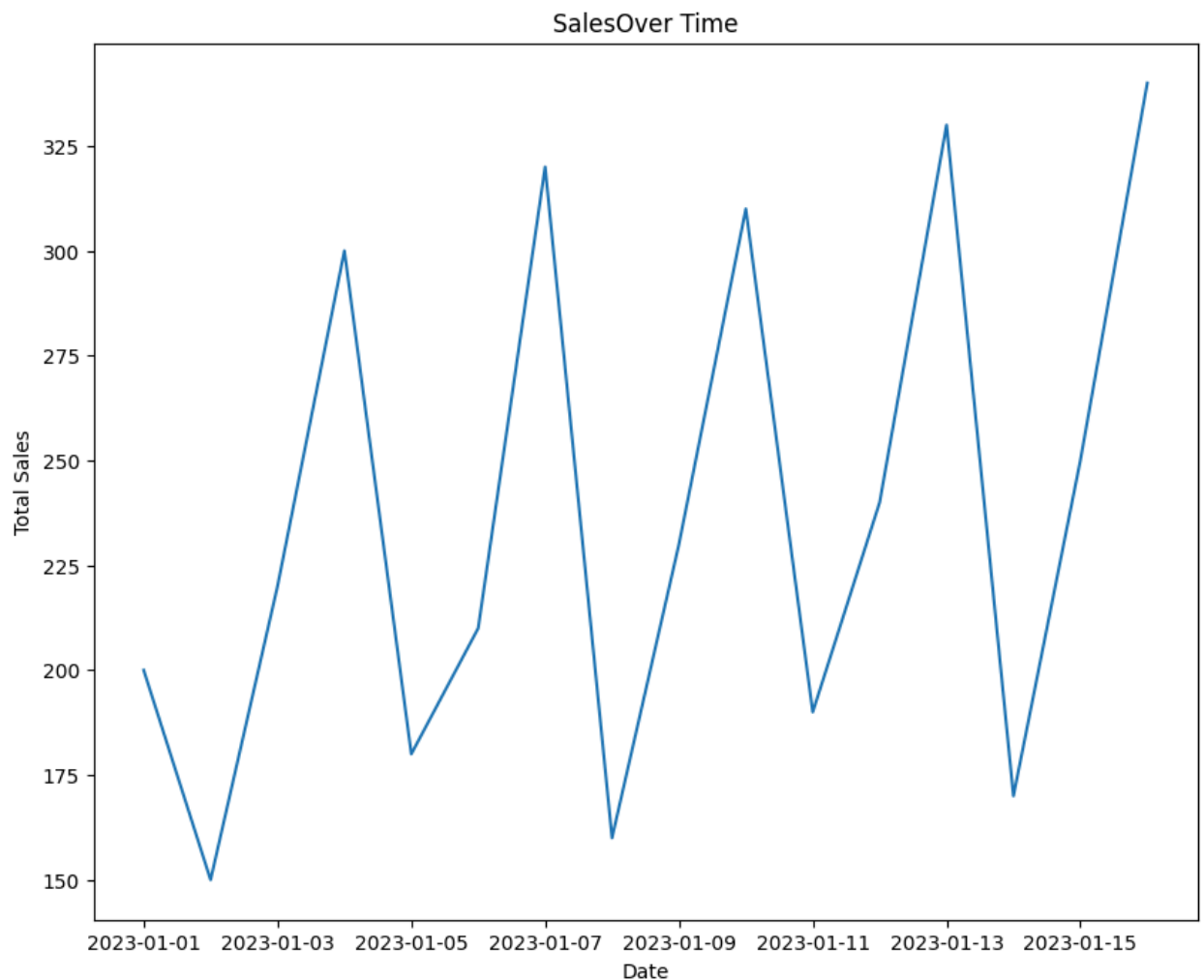
```
product_summary = df.groupby('Product').agg({'Sales': 'sum', 'Quantity': 'sum'}).
print(product_summary)
```

	Product	Sales	Quantity
0	Product A	1350	33
1	Product B	850	17
2	Product C	1600	36

```
plt.figure(figsize=(15, 8))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
```



```
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
plt.figure(figsize=(10,8))
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()
```



```
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product', aggregationfunc='sum')
print(pivot_table)
```

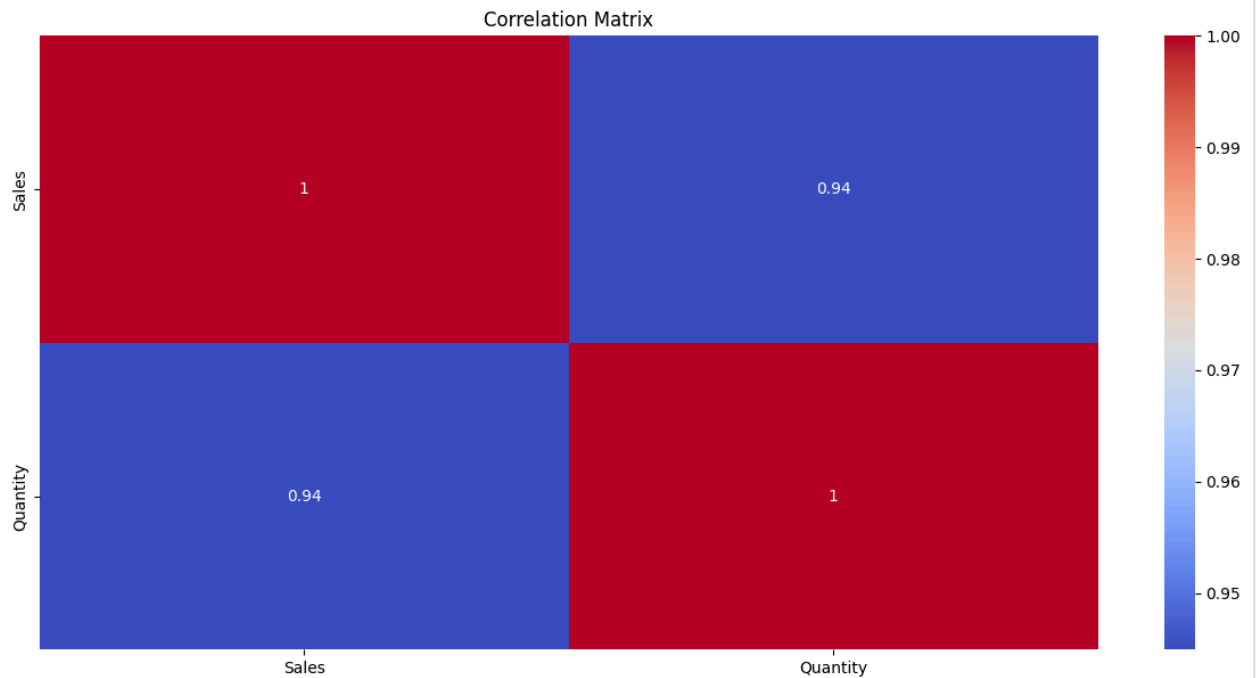
Product	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

```
/tmp/ipython-input-389367660.py:1: FutureWarning: The provided callable <function
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product', aggregationfunc='sum')
```

```
cor_mat=df[['Sales', 'Quantity']].corr()
print(cor_mat)
```

	Sales	Quantity
Sales	1.000000	0.944922

```
plt.figure(figsize=(15,7))  
sns.heatmap(cor_mat, annot=True, cmap='coolwarm')  
plt.title('Correlation Matrix')  
plt.show()
```



Result:

Hence, a Python program for **data preprocessing and visualization** of sales data has been successfully implemented, effectively cleaning, summarizing, and visualizing key insights from the dataset.